

Design Justification Report

QKT Matrix-Multiplication Hardware Accelerator — M4 Final

Pratibha Munnangi · pratibha@pdx.edu · ECE 410/510, Spring 2026 · Prof. Teuscher

1. Problem and Motivation

Transformer self-attention dominates compute cost in modern large language models. The QKT matrix multiplication — the first step in computing attention scores $\text{Attention}(Q,K,V) = \text{softmax}(QKT/\sqrt{d_k})V$ — accounts for over 60% of total FLOPs in BERT-base inference per forward pass.

M1 profiling on an Intel i5-1235U (4.4 GHz, 10 cores, AVX2) at the operating point $B=8$, $H=4$, $T=64$, $d_{\text{head}}=16$ measured a median wall-clock time of 0.981 ms and a throughput of 4.27 GFLOP/s for the QKT kernel alone. Despite this, the CPU operates at only 2.4% of its 176 GFLOPS theoretical peak. The gap exists because the kernel is memory-bandwidth-limited at small workload sizes, and because the general-purpose CPU expends energy on instruction decode, branch prediction, and multi-level cache management that custom silicon can eliminate.

This project designs, verifies, and synthesizes a standalone QKT accelerator chiplet targeting the sky130 130 nm open-source PDK via OpenLane 2. The goal is to demonstrate the architectural principles of custom-silicon acceleration: matched dataflow, operand reuse, and precision specialization. The system-level block diagram is shown in Figure 1 (see report/figures/block_diagram.png).

2. Roofline Analysis

The roofline model bounds performance as: $\text{Performance} = \min(\text{Peak Compute}, \text{Arithmetic Intensity} \times \text{Peak Bandwidth})$. Figure 2 (see report/figures/roofline_final.png) plots the complete roofline showing all four design revisions.

M4 v2 hardware parameters at nom_tt (81 MHz)

Parameter	Value	Derivation
Clock frequency	~81 MHz	WNS -4.36 ns at 8 ns target → closes at ~12.4 ns
Peak compute	2.59 GFLOPS	16 PEs × 2 FLOP/cycle × 81 MHz
AXI-Stream bandwidth	1.30 GB/s	16-bit bus × 81 MHz
Ridge point	2.0 FLOP/byte	Peak compute / Peak bandwidth (invariant to fmax)

Arithmetic intensity across design revisions

Design	AI (FLOP/byte)	Regime	Achievable @ fmax
M3 V5 (33 MHz, D=4, combinational)	0.5	Memory-bound	~0.03 GFLOPS
M4 v1 (100 MHz, D=4 scope)	0.5	Memory-bound	~0.65 GFLOPS

M4 v2 (81 MHz, D_total=16, no P0)	1.6	Memory-bound	~2.1 GFLOPS
M4 v2 (81 MHz, D_total=16 + P0 reuse)	≈3.5	Compute-bound ★	~2.59 GFLOPS (peak-limited)

Without d-chaining (D=4 scope), the host issues four separate chip invocations per output tile, re-streaming Q and K each time, collapsing AI to 0.5 FLOP/byte — well left of the 2.0 FLOP/byte ridge. The M4 architectural pass introduces d-dimension chaining over D_total=16: a single invocation covers the full d_head, raising AI to 1.6 FLOP/byte. Combined with P0 Q-row reuse (Q amortized across four K-tiles per row-band), system-level AI reaches ~3.5 FLOP/byte — crossing the ridge into compute-bound territory. As shown in Figure 2, this moves the M4 v2 operating point to the right of the ridge, meaning the 16-PE array is now the bottleneck rather than the AXI-Stream interface. The analysis directly shaped the d-chaining architectural decision described in Section 4.

3. Precision and Data Format

The design uses FP16 (IEEE 754 binary16) for input operands and FP32 (binary32) for accumulation throughout the pipeline. This mixed-precision choice is justified on three grounds.

FP16 operands: The M2 precision analysis verified that $\text{FP16} \times \text{FP16} \rightarrow \text{FP32}$ products are bit-exact for BERT-base embedding value ranges. Subnormals and NaN/Inf are flush-to-zero on input, simplifying the multiplier with negligible model-accuracy cost at typical embedding distributions.

FP32 accumulation: Accumulating 16 FP16 products risks up to 10–12 ULP error from repeated alignment shifts. FP32 accumulation preserves full product precision and matches the Python bit-exact reference for all 256/256 integration test cases in `tb_top.sv`.

V5 carry-out bug fix: The original combinational `fp32_adder` flushed same-sign equal-magnitude sums to zero when the 24-bit normalized result collapsed entirely into the carry-out bit (e.g., $0.03125 + 0.03125 = 0.0625$). The fix gates the all_zero flush on `!carry_out`. This fix is present in `fp32_adder_pipelined.sv` and was verified across 272 directed and random vectors in the standalone adder regression.

4. Dataflow and Architecture

Output-stationary dataflow

The 4×4 systolic array uses output-stationary dataflow. Each $\text{PE}[i][j]$ holds its partial sum in a persistent `tile_acc` register across all $N_BLOCKS=4$ d-blocks of a tile. Q operands enter the array row-wise from the left and propagate rightward; K operands enter column-wise from the top and propagate downward. A diagonal-skew driver in `top.sv` introduces per-row and per-column delays so $\text{PE}[i][j]$ receives Q row i and K row j simultaneously, computing $C[i][j] = Q[i,:] \cdot K[j,:]$ over $d_head=16$ dimensions.

Output-stationary minimizes partial-sum traffic: each output element is written exactly once after all inner-dimension products are accumulated. For QKT where the $N \times N$ output is large relative to d_head , this is the optimal dataflow choice for minimizing write traffic.

D-dimension chaining

Rather than requiring the host to issue four calls for `d_head=16` (one per `D=4` block), the M4 PE accumulates over `N_BLOCKS=4` consecutive `D=4` blocks via a persistent `tile_acc` register. After each block's tree-reduce produces `block_sum`, the PE launches a chain-add (`tile_acc + block_sum`) through the same pipelined adder. The top.sv FSM asserts `block_first` on the first block (seeding `tile_acc` directly) and `block_last` on the fourth (routing `tile_acc` to `acc_out`). This is the key change from M3 that raises AI from 0.5 to ~3.5 FLOP/byte with P0 reuse.

3-stage pipelined FP32 adder

The M3 V5 combinational adder had a ~58-level critical path, limiting `fmax` to 33 MHz. The M4 adder splits this into three pipeline stages of ~18 levels each: S1 (alignment shift + mantissa sign-extend), S2 (leading-zero count + normalize-shift barrel chain), S3 (round + output mux). This raises `fmax` to ~81 MHz — a 2.5× improvement. The adder is shared between the tree-reduce and the chain-add phase within each PE.

P0 Q-row reuse and P1 double-buffering

P0 retains the Q-row buffer across the four K-tiles of each output row-band, reducing Q AXI-Stream traffic by 4×. P1 uses dual-bank Q/K buffers so the load FSM can stream the next tile while the compute FSM processes the current tile, hiding transfer latency behind compute.

5. Hardware Interface

Interface	Width	Purpose
AXI4-Lite slave	32-bit (4 CSRs)	VERSION read, CONFIG write (N,D), CTRL.START, STATUS.DONE
AXI4-Stream slave	16-bit	Q and K FP16 operand streaming (1.30 GB/s @ 81 MHz)
AXI4-Stream master	32-bit	C FP32 result streaming (16 values per tile)

At 81 MHz the AXI4-Stream slave delivers 1.30 GB/s. With `D_total=16`, each tile invocation streams 64 Q beats and 64 K beats (128 total), amortizing AXI overhead across the full dot product. Without d-chaining (M3, `D=4` scope), four invocations were required per tile — the interface dominated latency and AI=0.5 placed the design memory-bound. With d-chaining and P0 Q-row reuse, AI rises to ~3.5 FLOP/byte, crossing the 2.0 FLOP/byte ridge: the design is interface-free and compute-bound.

6. Verification

Verification follows a three-level strategy building from standalone unit tests to full-chip AXI integration. Figure 3 (see report/figures/final_waveform.png) shows an annotated timing diagram of one complete tile transaction from the final simulation.

Level	Testbench	Vectors	Result
Adder standalone	tb_fp32_adder_pipelined.sv	272 (directed + random)	PASS 272/272
PE d-chain (unit)	tb_core_pe_chain.sv	74 (1/2/4 d-blocks, corners)	PASS 74/74

Full chip (integration)	tb_top.sv	256 (16 tiles × 16 outputs, d_head=16)	PASS 256/256
-------------------------	-----------	--	--------------

The adder testbench drives 272 bit-exact vectors against a Python emulator (fp32_adder_ref.py) mirroring the RTL gate-for-gate including the carry-out bug fix. Vectors cover same-sign equal-magnitude (V5 bug path), catastrophic cancellation, large exponent-difference pairs, and 256 random pairs.

The PE chain testbench (tb_core_pe_chain.sv) verifies d-chain accumulation using a linear-reference Python script (pe_chain_vector_gen.py). Cases include 1-block, 2-block, and 4-block tiles plus cancellation and small-magnitude corners. Bit-exact match is required; all 74/74 pass.

The integration testbench (tb_top.sv) streams d_head=16 Q and K operands via AXI4-Stream for a 16×16 logical QKT computation (16 tiles of 4×4). Outputs are compared against an independent FP16-quantized Python reference with 0.01 absolute tolerance. 256/256 outputs match. The simulation log is committed at project/m4/sim/final_run.log.

Key verification lesson: Two bugs invisible at M2 unit-test level emerged during M3 integration: (1) accumulator state bleed-through across back-to-back tiles without a tile-boundary reset, and (2) the FP32 adder equal-magnitude flush-to-zero. Both required multi-tile integration stimulus to trigger. The lesson: unit tests with clean initial state cannot catch state-persistence or boundary-condition bugs.

7. Synthesis Results

Synthesis was performed using OpenLane 2 targeting the sky130_fd_sc_hd standard-cell library at the nom_tt_025C_1v80 corner (nominal temperature and voltage). The 8 ns clock target was used with a 1500×1500 μm die. Reports are committed at project/m4/synth/.

Metric	M3 V5 (baseline)	M4 v1 (pipelined)	M4 v2 FINAL (d-chain)
SDC clock target	30 ns (33 MHz)	8 ns	8 ns
WNS nom_tt	+9.06 ns (closed)	-1.78 ns	-4.36 ns
Effective fmax	33 MHz	~100 MHz	~81 MHz
Cell instances	~33,000	63,508	85,545
Cell area	~134K μm^2	665K μm^2	928K μm^2
Die area	0.32 mm ²	1.21 mm ²	2.25 mm ²
Total power (nom_tt)	177 mW	114 mW	212 mW
Sequential power	3.2 mW (1.8%)	52.0 mW (46%)	73.3 mW (34.6%)
Combinational power	171 mW (96.7%)	17.5 mW (15%)	78.5 mW (37.1%)
Clock power	~2 mW (1.5%)	44.0 mW (38%)	60.0 mW (28.3%)
DRC violations	0	0	0
LVS	Clean	Clean	Clean

Critical path: The worst setup path in M4 v2 runs through the S2 stage of fp32_adder_pipelined inside PE[3][2] — specifically the LZC and variable left-shift (normalize-shift) barrel chain, ~18 logic levels. WNS = -4.36 ns at 8 ns target, establishing effective fmax at ~81 MHz. A 4-stage

pipeline split separating LZC from the left-shift barrel would reduce the critical path to ~12 levels and push fmax to ~130 MHz.

Area breakdown: The 85,545-cell design is dominated by the 16 pipelined adders (~270 pipeline registers each = ~4,320 FP32 flops per adder) and d-chain state (tile_acc registers, block control logic). The 29% area increase from v1 to v2 is attributable to the N_BLOCKS=4 d-chain state per PE.

Power analysis: Power increased from 114 mW (v1) to 212 mW (v2). The increase is driven by a larger 2.25 mm² die (vs 1.21 mm²) adding wire capacitance, which shows in the 37% combinational power share. The sequential share fell from 46% to 35% as the clock tree and wire capacitance grew.

8. Benchmark Results

The primary benchmark uses the M1 operating point: B=8, H=4, T=64, d_head=16, 4,194,304 FLOPs, 8,192 total 4×4 tiles. Wall-clock is derived from the simulation cycle count (2,254 cycles for 16 tiles from tb_top.sv) scaled to the full workload at each design's fmax. All raw data is in project/m4/bench/benchmark_data.csv.

Metric	M1 SW (CPU)	M3 V5	M4 v1	M4 v2 FINAL
fmax	4.4 GHz	33 MHz	100 MHz	81 MHz
AI (FLOP/byte)	—	0.5	0.5	~3.5 (compute-bound)
Throughput	4,270 MFLOPS	9.9 MFLOPS	90.9 MFLOPS	294 MFLOPS
Wall-clock (M1 wkld)	0.981 ms	424 ms	46.2 ms	14.2 ms
Speedup vs M1	1.0×	0.002× (431× slower)	0.021× (47× slower)	0.069× (14.5× slower)
vs Naive Python	28× faster	0.4× slower	3.3× faster	10.6× faster
vs M3 V5	—	1.0×	9.2×	29.8×
Power	15 W (TDP)	177 mW	114 mW	212 mW
Energy/call	14.7 mJ	75 mJ	5.26 mJ	3.02 mJ
Energy vs M1	1.0×	5.1× worse	2.8× better	4.9× better

M4 v2 is 14.5× slower than the M1 NumPy baseline in raw throughput. This gap is expected: (1) a 44× clock disadvantage (4.4 GHz CPU vs 81 MHz sky130 ASIC); (2) the M1 workload fits entirely in CPU L1/L2 cache while the chip streams through AXI-Stream; (3) AVX2 delivers 80 FP32 MACs/cycle vs the chip's 16 scalar MAC PEs. The spec requires: "If your accelerator is slower, say so and explain why" — this is the explanation.

M4 v2 uses 4.9× less energy per call than the CPU baseline (3.02 mJ vs 14.7 mJ). This is the genuine accelerator advantage: purpose-built hardware eliminates instruction decode, branch prediction, and cache hierarchy overhead that a general-purpose CPU cannot avoid. Against naive Python (27.6 MFLOPS), M4 v2 is 10.6× faster. The strongest story is the trajectory: M3 V5 took 424 ms at 75 mJ per call; M4 v2 takes 14.2 ms at 3.02 mJ — a 29.8× throughput improvement and 24.8× energy improvement.

9. What Did Not Work

M1 fmax projection vs sky130 actuals

The M1 milestone projected 500 MHz and a 16×16 array based on FPGA synthesis results, yielding 256 GFLOPS peak. sky130 standard-cell logic closes at 30–100 MHz for paths of this depth. Applying FPGA numbers to sky130 without process-specific calibration was the project's most significant early error. The lesson: calibrate projected fmax to the target process node with a small benchmark circuit before committing to array size or operating-point projections.

Timing: 8 ns target did not close

M4 v2 shows WNS -4.36 ns at the 8 ns target, establishing effective fmax at ~81 MHz. The bottleneck is the S2 stage of fp32_adder_pipelined (LZC + left-shift barrel chain, ~18 logic levels). A 4-stage adder split separating LZC from the variable left-shift would cut the critical path to ~12–14 levels and push fmax to ~130 MHz. This fix was not implemented due to verification budget: a 4th pipeline register shifts chain-add capture timing and requires re-verification across all three testbench levels.

Accumulator-in-loop hazard when pipelining the adder

The original M4 plan was to pipeline the M3 V5 accumulator-in-loop PE ($\text{acc_out} += \text{a_in} \times \text{b_in}$ each cycle). Pipelining the adder introduces a data hazard: the next product arrives before the previous accumulation exits the 3-stage pipeline. A naive "+2 cycle ARRAY_LAT" fix does not resolve the dependency. The correct fix — buffering all four products and reducing after collection — was implemented as the tree-reduce PE. This doubled RTL effort but produced a hazard-free design.

v2 power higher than v1

M4 v2 (212 mW) uses more power than M4 v1 (114 mW). The 1500×1500 μm die required to avoid routing congestion with ~85K cells produces longer wires, increasing switching capacitance. Combinational power rose from 15% to 37% of total. A hierarchical floorplan with explicit PE macro placement constraints would reduce routing wire length and recover the power advantage.

SS-corner timing closure

The slow-speed corner (nom_ss_100C_1v60) fails setup and hold checks in all revisions. SS imposes ~1.6× longer cell delays than nom_tt. Closing SS timing requires targeting ~50 MHz or a 4-stage adder pipeline. This was deferred since nom_tt is the primary academic characterization corner for sky130 chiplet projects.

16×16 array impractical on sky130

The M1 projection assumed a 16×16 PE array. Scaling to 16×16 would require over 1M cell instances and a die exceeding 20 mm², beyond the practical OpenLane 2 academic flow limit. Production deployment on a 28 nm process with the same RTL would achieve 1–2 GHz and a 16×16 array would deliver 50–100 GFLOPS, exceeding the CPU baseline. The 4×4 design on sky130 demonstrates the architectural principles at a scale tractable for a semester project.

--- End of Report ---